Guía teórica

CRUD de Tareas con ASP.NET Core en .NET 10

Controladores vs. Minimal API, inyección de dependencias, validación, ProblemDetails y OpenAPI

Solución TareasBackend · Octubre 2026

Índice

1. Qué construimos y por qué **3**
2. Fundamentos de la plataforma .NET **3**
2.1 Runtime, SDK y versiones 3
2.2 global.json 3
2.3 Solución y proyectos 4
3. HTTP, REST y el patrón CRUD **4**
3.1 Recursos y verbos 4
3.2 Códigos de estado 5
3.3 JSON y serialización 5
4. Arquitectura de la solución **5**
4.1 Por qué una librería compartida 5
4.2 Sin capa de repositorio: una decisión consciente 5
4.3 Inyección de dependencias 6
5. El modelo de datos **6**
5.1 Record frente a clase 6
5.2 Entidad y DTO: por qué dos tipos 7
6. Arranque y pipeline de ASP.NET Core **7**
6.1 Dos fases: servicios y middleware 7
6.2 Qué es un middleware 8
6.3 Entornos 8
7. Implementación con controladores **8**
7.1 Piezas clave 8
7.2 Model binding 9
8. Implementación con Minimal API **9**
8.1 Piezas clave 9
8.2 Validación integrada en .NET 10 10
9. Controladores frente a Minimal API **10**
10. Validación y manejo de errores **11**
10.1 DataAnnotations 11
10.2 ProblemDetails (RFC 9457) 11
10.3 Qué código devolver 11
11. OpenAPI y herramientas de prueba **11**
11.1 El documento OpenAPI 11
11.2 Archivos .http 12
12. Limitaciones y cómo evolucionar **12**
12.1 Concurrencia 12
12.2 Persistencia 12
12.3 Siguientes pasos razonables 12
13. Cómo ejecutar y probar **13**
14. Glosario **13**

1. Qué construimos y por qué

Este ejercicio construye un backend mínimo pero completo: dos Web API en ASP.NET Core que exponen el mismo CRUD (Create, Read, Update, Delete) sobre un listado de tareas guardado en memoria. Es el equivalente conceptual al ejemplo WeatherForecast que generan las plantillas de .NET, pero con operaciones de escritura además de lectura.

La particularidad es que la misma funcionalidad se implementa dos veces, con los dos estilos que ofrece ASP.NET Core:

- **Controladores** (proyecto Tareas.ControllersApi): el estilo clásico basado en clases que heredan de ControllerBase y atributos de enrutamiento.
- **Minimal API** (proyecto Tareas.MinimalApi): el estilo introducido en .NET 6, basado en lambdas o métodos estáticos registrados directamente sobre la aplicación.

Ambos proyectos comparten el modelo y los datos iniciales a través de una librería de clases (Tareas.Core). Comparar las dos implementaciones lado a lado es la mejor forma de entender qué hace el framework por nosotros en cada caso y qué decisiones de diseño hay detrás.

> **Objetivo didáctico.** Al terminar esta guía deberías poder explicar qué ocurre desde que un cliente envía una petición HTTP hasta que recibe una respuesta JSON, y por qué cada pieza del código está donde está.

2. Fundamentos de la plataforma .NET

2.1 Runtime, SDK y versiones

.NET es una plataforma de desarrollo multiplataforma y de código abierto. Conviene distinguir tres conceptos:

- **Runtime:** el motor que ejecuta las aplicaciones (CoreCLR). Incluye el recolector de basura, el compilador JIT y la librería base.
- **SDK:** las herramientas de desarrollo: compilador de C#, MSBuild, la CLI dotnet y las plantillas de proyecto. El SDK siempre incluye un runtime.
- **Target framework:** la versión de la plataforma contra la que compila un proyecto. En este ejercicio es net10.0, declarado en cada archivo .csproj.

.NET 10 es una versión LTS (Long Term Support) publicada en noviembre de 2025, con tres años de soporte. Incluye C# 14 y ASP.NET Core 10.

2.2 global.json

En la raíz de la solución hay un archivo global.json que fija la versión del SDK a utilizar. Es importante cuando en la máquina conviven varios SDK (en este caso también hay una preview de .NET 11): sin este archivo, la CLI usaría el SDK más reciente, incluidas las previews, y el resultado podría variar entre equipos.

```
{
  "sdk": {
    "version": "10.0.402",
```

```
    "rollForward": "latestPatch"
  }
}
```

La política `latestPatch` permite usar cualquier parche 10.0.4xx igual o superior al indicado, pero nunca saltar a una versión mayor.

2.3 Solución y proyectos

Una solución agrupa varios proyectos que se construyen juntos. .NET 10 genera por defecto el formato .slnx, una versión XML limpia y legible del antiguo .sln. Cada proyecto es un archivo .csproj que declara el tipo de proyecto (SDK), el target framework, los paquetes NuGet y las referencias a otros proyectos.

Proyecto	SDK	Función
Tareas.Core	Microsoft.NET.Sdk	Librería de clases: modelo Tarea, DTO TareaRequest y datos iniciales.
Tareas.ControllersApi	Microsoft.NET.Sdk.Web	Web API con controladores. Puerto http 5296.
Tareas.MinimalApi	Microsoft.NET.Sdk.Web	Web API con Minimal API. Puerto http 5223.

El SDK `Microsoft.NET.Sdk.Web` añade automáticamente las referencias a ASP.NET Core y los usings implícitos del espacio web, por eso no hace falta declararlos en el código.

3. HTTP, REST y el patrón CRUD

3.1 Recursos y verbos

REST (Representational State Transfer) es un estilo arquitectónico en el que la API expone recursos identificados por URL, y las operaciones sobre ellos se expresan con los verbos estándar de HTTP. En nuestro caso el recurso es la colección de tareas, accesible en /api/tareas, y cada tarea individual en /api/tareas/{id}.

Verbo	Ruta	Operación CRUD	Idempotente	Respuesta correcta
GET	/api/tareas	Read (todas)	Sí	200 OK + lista JSON
GET	/api/tareas/{id}	Read (una)	Sí	200 OK + objeto, o 404
POST	/api/tareas	Create	No	201 Created + Location + objeto
PUT	/api/tareas/{id}	Update (reemplazo)	Sí	200 OK + objeto, o 404
DELETE	/api/tareas/{id}	Delete	Sí	204 No Content, o 404

Idempotencia significa que repetir la misma petición produce el mismo estado en el servidor. PUT y DELETE lo son: actualizar dos veces con el mismo cuerpo deja la tarea igual, y borrar dos veces deja

la tarea igualmente ausente (aunque la segunda vez responda 404). POST no lo es: cada llamada crea una tarea nueva con un Id distinto.

3.2 Códigos de estado

El código de estado es la forma en que el servidor comunica el resultado sin que el cliente tenga que interpretar el cuerpo. Los que usa este ejercicio:

- **200 OK:** la operación tuvo éxito y el cuerpo contiene el recurso.
- **201 Created:** se creó un recurso. La cabecera Location indica dónde consultarlo (por ejemplo http://localhost:5223/api/tareas/6).
- **204 No Content:** éxito sin cuerpo. Es lo habitual tras un DELETE.
- **400 Bad Request:** la petición es inválida (por ejemplo, título vacío). El servidor explica qué falló.
- **404 Not Found:** el recurso pedido no existe.
- **500 Internal Server Error:** fallo no controlado en el servidor. Nunca debería ser una respuesta deliberada.

3.3 JSON y serialización

ASP.NET Core serializa los objetos de respuesta con System.Text.Json. Por defecto convierte los nombres de propiedad a camelCase: la propiedad Titulo de C# se emite como titulo. En la deserialización ocurre el proceso inverso, y la comparación es insensible a mayúsculas.

```
{ "id": 1, "titulo": "Comprar leche", "completada": false }
```

4. Arquitectura de la solución

4.1 Por qué una librería compartida

Las dos APIs son intercambiables desde el punto de vista del cliente: misma ruta, mismo JSON, mismos códigos. Para garantizarlo, el modelo de datos vive una sola vez en Tareas.Core y los dos proyectos web lo referencian. Si mañana la tarea necesita una fecha de vencimiento, se añade en un único sitio.

La librería es deliberadamente pequeña: no contiene lógica de acceso a datos ni servicios. Solo tres archivos:

- `Tarea.cs`: el modelo de dominio, un record inmutable.
- `TareaRequest.cs`: el DTO (Data Transfer Object) que el cliente envía en POST y PUT, con sus reglas de validación.
- `TareasSeed.cs`: un método que devuelve la lista inicial de tareas.

4.2 Sin capa de repositorio: una decisión consciente

El patrón Repositorio abstrae el acceso a datos tras una interfaz (IRepository) para poder cambiar la implementación (memoria, SQL, API externa) sin tocar quien lo consume. Es valioso cuando existe esa variabilidad o cuando se quiere aislar la lógica para pruebas unitarias.

En este ejercicio se ha prescindido de él a propósito. Los controladores y los endpoints reciben directamente una List<Tarea> y operan sobre ella con LINQ. Esto mantiene el foco en lo que se quiere aprender (la mecánica de las APIs) y evita capas que, en un ejemplo de este tamaño, serían ceremonia sin beneficio.

> **Cuándo sí tiene sentido.** Si la lista pasara a una base de datos, si hubiera varias fuentes de datos o si se necesitaran tests unitarios que sustituyan el almacenamiento, introducir un repositorio o un servicio sería el paso natural.

4.3 Inyección de dependencias

ASP.NET Core incorpora un contenedor de inyección de dependencias (DI). Durante el arranque se registran los servicios en builder.Services, y el framework los entrega automáticamente a quien los declare como parámetro: el constructor de un controlador o los parámetros de un handler de Minimal API.

```
// Program.cs (ambas APIs)
builder.Services.AddSingleton(TareasSeed.Crear());
```

Esta única línea registra una instancia de List<Tarea> ya poblada. A partir de ahí, cualquier parámetro de tipo List<Tarea> recibe esa misma instancia.

Ciclos de vida

Ciclo de vida	Se crea	Uso típico
Singleton	Una vez por aplicación	Caches, configuración, estado compartido en memoria (nuestra lista).
Scoped	Una vez por petición HTTP	DbContext de Entity Framework, unidades de trabajo.
Transient	Cada vez que se solicita	Servicios ligeros sin estado.

La lista es Singleton porque queremos que las tareas creadas en una petición sean visibles en las siguientes. Si fuera Scoped, cada petición recibiría una lista nueva con los cinco elementos iniciales y los cambios se perderían al instante.

5. El modelo de datos

5.1 Record frente a clase

```
public sealed record Tarea(int Id, string Titulo, bool Completada);
```

Un record posicional genera automáticamente un constructor, propiedades de solo lectura (init), igualdad por valor, ToString legible y la expresión with para crear copias modificadas. Es ideal para

modelos inmutables: una Tarea no se modifica, se sustituye por otra con los nuevos valores, como hace el método Update.

```
var tarea = new Tarea(id, request.Titulo, request.Completada);
tareas[index] = tarea;   // se reemplaza el elemento, no se muta
```

5.2 Entidad y DTO: por qué dos tipos

TareaRequest existe porque el cliente no debe decidir el Id: lo asigna el servidor. Si POST aceptara directamente una Tarea, el cliente podría enviar un Id arbitrario y habría que ignorarlo o validarlo. Separar lo que entra (DTO) de lo que se almacena y devuelve (entidad) es una práctica estándar que evita exponer campos internos y permite reglas de validación específicas de la entrada.

```
public sealed class TareaRequest
{
    [Required, MinLength(1), MaxLength(200)]
    public string Titulo { get; init; } = string.Empty;

    public bool Completada { get; init; }
}
```

> **Lección aprendida durante el ejercicio.** La primera versión del DTO era un record posicional con los atributos escritos como [property: Required]. El binder de MVC detecta esa configuración y lanza una excepción en tiempo de ejecución (HTTP 500), porque en records posicionales exige que la validación se asocie al parámetro del constructor, no a la propiedad generada. Convertir el DTO en una clase con propiedad init resolvió el problema y funciona igual en ambos estilos.

6. Arranque y pipeline de ASP.NET Core

6.1 Dos fases: servicios y middleware

Program.cs tiene siempre la misma estructura en dos tiempos. Primero, con el builder, se registran servicios en el contenedor DI. Después, con app, se compone el pipeline de middleware y se mapean los endpoints.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();              // solo en la API de controladores
builder.Services.AddProblemDetails();
builder.Services.AddOpenApi();
builder.Services.AddSingleton(TareasSeed.Crear());

var app = builder.Build();

app.UseExceptionHandler();
app.UseStatusCodePages();
if (app.Environment.IsDevelopment()) app.MapOpenApi();
app.UseHttpsRedirection();
app.MapControllers();                           // o app.MapTareasEndpoints();
app.Run();
```

6.2 Qué es un middleware

Cada petición atraviesa una cadena de componentes, los middleware, en el orden en que se registran. Cada uno puede actuar antes de pasar la petición al siguiente y después de que éste devuelva la respuesta. Por eso el orden importa:

- **UseExceptionHandler** va primero para capturar cualquier excepción de los middleware posteriores y convertirla en una respuesta 500 con formato ProblemDetails.
- **UseStatusCodePages** añade un cuerpo ProblemDetails a las respuestas de error que llegan vacías (por ejemplo un 404 de ruta inexistente).
- **UseHttpsRedirection** redirige las peticiones http a https cuando el perfil de ejecución lo expone.
- **MapControllers / MapTareasEndpoints** registran los endpoints en el enrutador, que es el último eslabón.

6.3 Entornos

La variable ASPNETCORE_ENVIRONMENT (definida en launchSettings.json como Development) permite condicionar comportamiento. Aquí se usa para exponer el documento OpenAPI solo en desarrollo, evitando publicar la descripción completa de la API en producción.

7. Implementación con controladores

Un controlador es una clase que hereda de ControllerBase y agrupa acciones (métodos públicos) relacionadas con un recurso. El framework MVC se encarga de enrutar, enlazar parámetros, validar y formatear la respuesta.

```
[ApiController]
[Route("api/tareas")]
public sealed class TareasController(List<Tarea> tareas) : ControllerBase
{
    [HttpGet("{id:int}", Name = nameof(GetById))]
    public ActionResult<Tarea> GetById(int id) =>
        tareas.FirstOrDefault(t => t.Id == id) is { } tarea ? Ok(tarea) : NotFound();

    [HttpPost]
    public ActionResult<Tarea> Create([FromBody] TareaRequest request)
    {
        var nextId = tareas.Count == 0 ? 1 : tareas.Max(t => t.Id) + 1;
        var tarea = new Tarea(nextId, request.Titulo, request.Completada);
        tareas.Add(tarea);
        return CreatedAtRoute(nameof(GetById), new { id = tarea.Id }, tarea);
    }
}
```

7.1 Piezas clave

- **[ApiController]** activa comportamientos pensados para APIs: validación automática del modelo con respuesta 400, inferencia de [FromBody] para tipos complejos y respuestas de error en formato ProblemDetails.
- **[Route] y [HttpGet/Post/Put/Delete]** definen la ruta. La plantilla {id:int} incluye una restricción de ruta: si el segmento no es un entero, la ruta no coincide y se responde 404.
- **Constructor primario** (C# 12+): el parámetro tareas queda disponible en toda la clase sin declarar un campo. El contenedor DI lo rellena en cada petición.
- **ActionResult<T>** permite devolver tanto el tipo T (Ok) como resultados de otro código (NotFound) desde la misma acción, y documenta el tipo de éxito para OpenAPI.
- **CreatedAtRoute** construye la respuesta 201 y calcula la cabecera Location a partir del nombre de la ruta GetById y el Id generado.
- **[ProducesResponseType]** no cambia el comportamiento; enriquece el documento OpenAPI declarando qué códigos puede devolver cada acción.

7.2 Model binding

El model binding es el proceso por el que MVC convierte la petición HTTP en parámetros de C#. Los valores se buscan en la ruta (id), la query string, las cabeceras o el cuerpo (TareaRequest, deserializado desde JSON). Tras el binding, MVC ejecuta la validación por DataAnnotations y, con [ApiController], responde 400 automáticamente si hay errores, sin llegar a ejecutar la acción.

8. Implementación con Minimal API

Minimal API elimina la clase controlador: cada endpoint es un delegado registrado directamente sobre la aplicación o sobre un grupo de rutas. Todo lo que MVC hace mediante atributos y convenciones aquí se expresa con métodos de extensión y con el sistema de tipos.

```
var group = app.MapGroup("/api/tareas").WithTags("Tareas");
group.MapGet("/{id:int}", GetById).WithName("GetTareaById");
group.MapPost("/", Create).WithName("CreateTarea");

private static Results<Ok<Tarea>, NotFound> GetById(int id, List<Tarea> tareas) =>
    tareas.FirstOrDefault(t => t.Id == id) is { } tarea
        ? TypedResults.Ok(tarea)
        : TypedResults.NotFound();

private static CreatedAtRoute<Tarea> Create(TareaRequest request, List<Tarea> tareas)
{
    var nextId = tareas.Count == 0 ? 1 : tareas.Max(t => t.Id) + 1;
    var tarea = new Tarea(nextId, request.Titulo, request.Completada);
    tareas.Add(tarea);
    return TypedResults.CreatedAtRoute(tarea, "GetTareaById", new { id = tarea.Id });
}
```

8.1 Piezas clave

- **MapGroup** agrupa endpoints bajo un prefijo común y permite aplicar metadatos (etiquetas, filtros, autorización) a todos de una vez.

- **Inferencia de parámetros:** id se toma de la ruta porque coincide con el nombre del segmento; TareaRequest del cuerpo porque es un tipo complejo; List<Tarea> del contenedor DI porque está registrado como servicio. No hacen falta atributos.
- **TypedResults** devuelve tipos concretos (Ok<Tarea>, NotFound) en lugar de la interfaz IResult. Esto hace que el compilador conozca las respuestas posibles y que OpenAPI las documente sin anotaciones adicionales.
- **Results<T1, T2>** es una unión de tipos: declara que el handler devuelve exactamente uno de esos resultados. Sustituye a los [ProducesResponseType] de MVC.
- **WithName** da nombre al endpoint, necesario para que CreatedAtRoute pueda generar la URL del recurso creado.

8.2 Validación integrada en .NET 10

Históricamente Minimal API no validaba DataAnnotations: había que hacerlo a mano o con paquetes de terceros. .NET 10 incorpora validación nativa activada con una línea:

```
builder.Services.AddValidation();
```

Un generador de código en tiempo de compilación analiza los parámetros de los endpoints y produce el código de validación, sin reflexión en tiempo de ejecución. Si falla, responde 400 con el mismo formato ValidationProblemDetails que MVC.

9. Controladores frente a Minimal API

Aspecto	Controladores	Minimal API
Unidad de organización	Clase por recurso con varias acciones	Grupo de rutas con handlers (lambdas o métodos estáticos)
Enrutamiento	Atributos [Route], [HttpGet]...	Métodos MapGet, MapPost... sobre app o un grupo
Binding de parámetros	Convenciones + atributos [FromBody], [FromRoute]...	Inferencia por tipo y nombre; atributos opcionales
Documentar respuestas	[ProducesResponseType] y ActionResult<T>	TypedResults y Results<T1, T2>, inferidos por el compilador
Validación	Automática con [ApiController]	Automática con AddValidation() desde .NET 10
Filtros	Filtros de acción, resultado, excepción...	Endpoint filters (AddEndpointFilter)
Rendimiento de arranque	Mayor: descubre controladores por reflexión	Menor: todo se registra explícitamente; compatible con AOT
Cuándo elegirlo	APIs grandes, equipos acostumbrados a MVC, necesidad de filtros complejos	Microservicios, APIs pequeñas o medianas, compilación nativa AOT

No hay un ganador absoluto. Ambos estilos conviven en la misma aplicación si hace falta, comparten el pipeline, el contenedor DI, el enrutamiento y OpenAPI. Microsoft recomienda Minimal API para proyectos nuevos, pero los controladores siguen siendo plenamente soportados.

10. Validación y manejo de errores

10.1 DataAnnotations

Los atributos del espacio System.ComponentModel.DataAnnotations declaran reglas sobre las propiedades: [Required] exige valor no nulo ni vacío, [MinLength] y [MaxLength] acotan la longitud. Son declarativos: la propiedad dice qué es válido, y el framework decide cuándo comprobarlo.

10.2 ProblemDetails (RFC 9457)

Cuando algo falla, la API no devuelve texto libre sino un documento JSON estandarizado llamado Problem Details. Su formato está definido por la RFC 9457 y lo entienden la mayoría de clientes y herramientas:

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": { "Titulo": ["The Titulo field is required."] },
  "traceId": "00-a426db54b826826f486b5af476b704e0-99e46357fd6fc3d2-00"
}
```

El campo traceId enlaza la respuesta con los logs del servidor, lo que facilita diagnosticar un fallo concreto. AddProblemDetails registra el servicio que genera estos documentos, y UseExceptionHandler y UseStatusCodePages lo usan para los errores no controlados y las respuestas de error vacías.

10.3 Qué código devolver

- **400** cuando la culpa es del cliente y la petición es sintáctica o semánticamente inválida.
- **404** cuando la petición es válida pero el recurso no existe. No es un error del cliente ni del servidor; es información.
- **500** solo para fallos inesperados. Si ves un 500 reproducible, hay un bug que corregir, como ocurrió con el record y sus atributos.

11. OpenAPI y herramientas de prueba

11.1 El documento OpenAPI

OpenAPI (antes Swagger) es una especificación para describir APIs HTTP en JSON o YAML: rutas, verbos, parámetros, esquemas de cuerpo y respuestas. ASP.NET Core 10 genera ese documento de forma nativa con el paquete Microsoft.AspNetCore.OpenApi, disponible en /openapi/v1.json.

A partir de él se pueden generar clientes tipados en cualquier lenguaje, importar la API en Postman o Insomnia, o montar una interfaz interactiva con Swagger UI o Scalar. El documento se construye a partir de los metadatos que ya hemos visto: los atributos [ProducesResponseType] en controladores y los tipos de TypedResults en Minimal API.

11.2 Archivos .http

Cada proyecto incluye un archivo .http con peticiones listas para ejecutar desde Visual Studio o VS Code (extensión REST Client). Es una forma ligera y versionable de documentar cómo se usa la API y de probarla sin herramientas externas.

```
### Crear
POST {{Tareas.MinimalApi_HostAddress}}/api/tareas
Content-Type: application/json

{
  "titulo": "Regar las plantas",
  "completada": false
}
```

12. Limitaciones y cómo evolucionar

12.1 Concurrencia

List<T> no es segura para accesos concurrentes. Kestrel, el servidor web de ASP.NET Core, atiende peticiones en paralelo, así que dos POST simultáneos podrían corromper la lista o generar el mismo Id. Para una demo es aceptable; para algo más serio hay tres caminos de menor a mayor esfuerzo:

1. Envolver las operaciones de escritura en un `lock` sobre la lista.
2. Sustituir la lista por `ConcurrentDictionary<int, Tarea>` y generar Ids con `Interlocked.Increment`.
3. Mover los datos a una base de datos con Entity Framework Core, donde la concurrencia la gestiona el motor.

12.2 Persistencia

Los datos viven en la memoria del proceso: al reiniciar la aplicación vuelven las cinco tareas iniciales. Además, cada API tiene su propia copia de la lista; crear una tarea en el puerto 5296 no la hace visible en el 5223. Compartir estado real entre dos procesos exige un almacenamiento externo (base de datos, Redis, archivo).

12.3 Siguientes pasos razonables

- Añadir un proyecto de pruebas con xUnit y WebApplicationFactory para probar los endpoints de extremo a extremo.
- Introducir Entity Framework Core con SQLite o SQL Server y, entonces sí, un repositorio o un servicio de aplicación.
- Añadir paginación y filtrado a GET /api/tareas (por ejemplo ?completada=false).
- Proteger la API con autenticación JWT y autorización por políticas.

- Montar Scalar o Swagger UI sobre el documento OpenAPI para explorar la API en el navegador.

13. Cómo ejecutar y probar

```
dotnet build
dotnet run --project src/Tareas.ControllersApi    # http://localhost:5296
dotnet run --project src/Tareas.MinimalApi        # http://localhost:5223
```

Con cualquiera de las dos en marcha, estas peticiones recorren el CRUD completo:

```
curl http://localhost:5223/api/tareas
curl http://localhost:5223/api/tareas/1
curl -X POST -H "Content-Type: application/json" \
     -d '{"titulo":"Regar las plantas","completada":false}' \
     http://localhost:5223/api/tareas
curl -X PUT -H "Content-Type: application/json" \
     -d '{"titulo":"Comprar leche y pan","completada":true}' \
     http://localhost:5223/api/tareas/1
curl -X DELETE http://localhost:5223/api/tareas/2
curl http://localhost:5223/openapi/v1.json
```

14. Glosario

Término	Definición
API	Interfaz de programación: contrato que expone funcionalidad a otros programas, aquí mediante HTTP.
CRUD	Las cuatro operaciones básicas sobre datos: Create, Read, Update, Delete.
DTO	Data Transfer Object. Tipo pensado solo para transportar datos entre cliente y servidor.
DI	Dependency Injection. Técnica por la que un componente recibe sus dependencias en lugar de crearlas.
Endpoint	Combinación de ruta y verbo HTTP que ejecuta un handler concreto.
Handler	Método o lambda que procesa una petición en Minimal API.
Kestrel	Servidor web multiplataforma incluido en ASP.NET Core.
Middleware	Componente del pipeline que procesa peticiones y respuestas en cadena.
Model binding	Conversión automática de la petición HTTP en parámetros tipados de C#.
OpenAPI	Especificación estándar para describir APIs HTTP de forma legible por máquinas.
ProblemDetails	Formato JSON estándar (RFC 9457) para describir errores HTTP.
Record	Tipo de C# con igualdad por valor y sintaxis concisa, ideal para datos inmutables.
REST	Estilo arquitectónico basado en recursos, URLs y verbos HTTP.
Singleton	Ciclo de vida DI con una única instancia para toda la aplicación.